

DATA 266: Generative AI, Fall 2026

Lab 1 Report

Character-Level Language Model Pretraining, Sentiment Classification,
and CycleGAN Image Style Transfer

Team 09 (PairProgramming_Team_09)

Shibin Thomas
Teammate (name to be added)

Repository: <https://github.com/Shibin-1020/Gen-AI---LAB-1>

Branch: shibin-lab1

October 2026

1 Team Ownership Statement

Shibin Thomas built the character-level GPT for Task 1, the mean-pooling baseline, TextCNN and BiGRU-with-attention classifiers for Task 2, and the CycleGAN for Task 3, including the Kaggle submission. Shibin also set up the shared TinyStories download, the shared metric modules for Tasks 1 and 2, the repository layout, the run manifests and the smoke tests.

Teammate (name to be added) built their own Task 1 language model, three Task 2 classifiers and Task 3 CycleGAN (details to be added).

Each member trained their own models from scratch in a personal folder for every task (task1_llm/<member>, task2_sentiment/<member>, task3_gan/<member>). To keep the results comparable we shared the raw TinyStories file and the generation prompts for Task 1, the official Yelp Polarity test split and a common metric module for Task 2, and the instructor's evaluation script for Task 3. The comparison tables and the analysis sections of this report were written together.

Every member folder follows the same layout: source code and the task notebook in `src/`, configuration files in `configs/`, model weights in `checkpoints/<run_id>/`, plots and samples in `outputs/<run_id>/`, and the files `metrics_report.csv`, `results.md` and `failure_analysis.md`. Unedited training logs are kept in `reproducibility/raw_logs/` and a manifest for each run (configuration, git commit, library versions, hardware and checkpoint hashes) in `reproducibility/manifests/`. Each figure and table below names the file it was taken from, so every number can be traced back to its log or checkpoint.

2 Task 1: GPT-Style Character-Level Language Model

The model is a decoder-only Transformer trained on characters from TinyStories [2]. All components (token and positional embeddings, layer normalisation, multi-head causal self-attention, the feed-forward blocks and the residual connections) were written by hand following Vaswani et al. [1]; no built-in Transformer or attention module of PyTorch is used, and a unit test (`tests/test_model.py`) checks this. Each member drew their own split from the cleaned story pool. Shibin's split contains 100,000 training and 10,000 validation stories (seed 266), and the character vocabulary of 90 symbols was built from the training split only. The network has 6 layers, a model width of 320, 8 attention heads and a context of 256 characters, about 7.5 million parameters in total.

2.1 Model Comparison

Table 1. Task 1 comparison of architecture, hyperparameters and metrics. Losses, perplexity, BPC and accuracy are computed on the best checkpoint; metric definitions follow the shared module `task1_llm/shared_eval/metrics.py`.

	Shibin Thomas (gpt_char_v1)	teammate GPT
Architecture	6 layers, d_model 320, 8 heads (head dim 40), FFN 4x with GELU, pre-LayerNorm, learned positions, tied input/output embeddings, context 256	Pending
Hyperparameters	AdamW (0.9, 0.95), weight decay 0.1, peak LR 0.0006 with 2% warm-up and cosine decay to 6e-05, batch 64 x 256, gradient clip 1.0, dropout 0.1, 10 epochs, bf16	Pending
Training cross-entropy (nats/char)	0.5594	Pending
Validation cross-entropy (nats/char)	0.5711	Pending
Validation perplexity	1.7701	Pending
Validation bits per character	0.8239	Pending
Training perplexity / BPC	1.7497 / 0.8071	Pending
Generalisation gap (val - train CE)	0.0116	Pending
Next-character accuracy (validation)	0.8173	Pending
Distinct-1 / 2 / 3 (sampled, T = 0.8)	0.2277 / 0.7048 / 0.9264	Pending
Repeated 4-gram rate (sampled / greedy)	0.0066 / 0.1373	Pending
Gradient norm mean / p99 / max	0.304 / 1.145 / 14.71	Pending
Clipped steps / loss spikes / NaN steps	1.34% / 0 / 0	Pending
Parameters (total / non-embedding)	7,509,120 / 7,398,400	Pending
Training throughput (characters/s)	569,917	Pending
Generation speed (characters/s)	255.0	Pending
Peak GPU memory (MB)	3,347	Pending
Training time (min) / epochs / best epoch	28.9 / 10 / 10	Pending
Hardware	NVIDIA GeForce RTX 5090	Pending
Run and checkpoint	gpt_char_v1_20261001-153007, best_model.pt (SHA-256 97647773d06f)	Pending

Source: `task1_llm/shibin_thomas/metrics_report.csv`

2.2 Training Behaviour

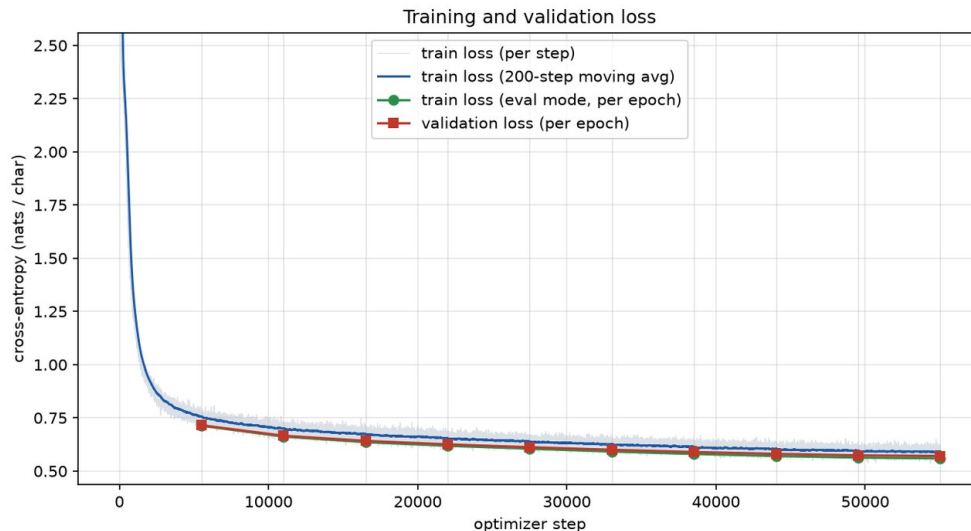


Figure 1. Task 1 training and validation loss (Shibin Thomas)
(task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007/loss_curves.png)

Table 2. Validation cross-entropy (nats/char) and next-character accuracy (%) after each epoch.

Epoch	1	2	3	4	5	6	7	8	9	10
Val. CE	0.715	0.666	0.642	0.625	0.612	0.600	0.590	0.581	0.574	0.571
Val. acc.	77.4	78.8	79.6	80.1	80.5	80.8	81.2	81.4	81.6	81.7

Source: reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007/epochs.csv

The training loss starts at 4.52, which is the loss of a uniform guess over 90 characters, and drops below 1.0 within the first 1,400 steps. After that the validation loss keeps falling every epoch, from 0.715 to 0.571, so the last epoch is also the best one. The gap between validation and training loss stays very small (0.012 nats at the end), which is expected with roughly 120 training characters per parameter. Training was stable: there were no NaN steps or loss spikes, and all of the 738 clipped steps happened during the learning-rate warm-up.

2.3 Discussion

Strengths. A small model of 7.5 million parameters, trained for 29 minutes on one GPU, already writes fluent TinyStories-style text with named characters, dialogue and simple cause and effect. It reaches a validation BPC of 0.824 and predicts the next character correctly 81.7% of the time. With temperature sampling the output is varied (Distinct-3 of 0.93 and almost no repeated 4-grams).

Weaknesses. Greedy decoding falls into loops, with a repeated 4-gram rate of 0.137. Because words are produced one character at a time, rare words are often misspelled, and sometimes spelled differently each time they appear. Stories also lose their thread once they grow beyond the 256-character context.

Limitations. Each model was trained once with a single seed. The validation loss was still decreasing at epoch 10, so the model is not fully converged. Distinct-n and the repetition rate measure diversity rather than story quality, and generation speed was measured without a key-value cache.

Next steps. We would train for more epochs or use a slightly larger model, extend the context to 512 characters, and compare top-k or nucleus sampling with a repetition penalty on the shared prompts. A key-value cache would speed up generation, and a sub-word tokenizer trained from scratch would remove most of the spelling errors.

2.4 Failure Analysis (Shibin Thomas)

We generated text for the 10 shared prompts, with one greedy and three sampled continuations ($T = 0.8$) of 500 characters each. The three clearest failures are shown below; the full set of 40 samples is in `outputs/<run_id>/samples.md`.

Case 1: Repetition (greedy decoding)

In a small house near the forest, there was a big tree. The tree was very high and the tree was very high. The tree was very high up in the tree. The tree was very high and the tree was very high.

One day, the tree saw a big tree. The tree was very high and the tree was very high. The tree was very high and the tree was very high. ...

The phrase “the tree was very” appears 16 times and the repeated 4-gram rate of this sample is 0.61, against 0.04 on average. Greedy decoding always takes the most likely character, so once a high-probability phrase has been written, starting it again is again the most likely continuation. Sampling, or a repetition penalty, would break the loop; the repeated 4-gram rate on the same prompts is the measure to check.

Case 2: Broken and inconsistent spelling ($T = 0.8$)

Tim was scared. He ran and ran until he came across a big lute. He saw a small tree which was very pretty. Tim wanted to take the lutter from him. He said yes, but he had to be careful.

The luter was big and boring. Tim saw that the lutte was scary and hurt. ...

The rare word “lute” is spelled four different ways (lute, lutter, luter, lutte). The model has no word list and rebuilds a rare word from frequent letter patterns each time it comes back. A lower temperature, longer training or sub-word tokens should reduce this.

Case 3: Loss of coherence ($T = 0.8$)

After school, Sam wanted to go to the zoo and see all the animals.

They all walked closer and held on tight. Sam thought it was tremometer than the diamond. ... Together, Jack and Sam started to fill their bucket with food. He was so proud of their work! ... Tom and Sam continued to throw their bucket until it was time to go hom

Each sentence is fluent, but the setting jumps from the zoo to a diamond and a bucket, two new friends (Jack and Tom) appear without introduction, and the pronouns stop matching. The continuation is 500 characters long while the model only sees the last 256, so the beginning of the story has literally dropped out of its context. A longer context or a deeper model would be the fix to test.

The teammate's failure cases are in their own `task1_llm/<member>/failure_analysis.md`.

2.5 Evidence

Evidence	Location in the repository
Configuration	task1_llm/shibin_thomas/configs/gpt_char_v1.yaml
Training log	reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007/train.log
Per-step and per-epoch logs	reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007
Run manifest	reproducibility/manifests/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007.json
Plots (loss, gradient norm, LR)	task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007
Generated samples	task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007/samples.md
Best checkpoint	task1_llm/shibin_thomas/checkpoints/gpt_char_v1_20261001-153007/best_model.pt
Results and design choices	task1_llm/shibin_thomas/results.md
Executed notebook	task1_llm/shibin_thomas/src/task1_gpt_tinystories.ipynb

3 Task 2: Sentiment Classification on Yelp Polarity

All Task 2 models classify a review as positive or negative and learn their word embeddings from random initialisation; no pretrained embeddings or language models are used. The official test split (38,000 reviews) is shared by the team. Each member carved a validation set out of the training split; Shibin used 50,000 reviews (seed 266) for early stopping and model selection. Preprocessing removes duplicates and reviews that also appear in the test set, lowercases the text, expands contractions, removes punctuation and stopwords while keeping negations and contrast words, and applies Snowball stemming. The vocabulary of 30,000 tokens is built from the training split only.

Shibin trained three models that share the same 128-dimensional embedding, so that differences in the results come from the architecture. The baseline averages the word embeddings of a review and feeds them to a small MLP. The first experimental model is a TextCNN [7] with filter widths 3, 4 and 5 and max pooling over time. The second is a two-layer bidirectional GRU with additive attention pooling [9].

3.1 Model Comparison

Table 3. Task 2 comparison on the official test split (n = 38,000). All models share: vocabulary 30K, maximum length 256 tokens (head and tail), 3% warm-up with cosine decay, weight decay 1e-4, gradient clipping 1.0, early stopping on validation macro-F1 (patience 2), threshold 0.5. Run yelp3_20261001-182849.

	Shibin: Baseline (mean pool)	Shibin: Exp. 1 (TextCNN)	Shibin: Exp. 2 (BiGRU + attn.)	teammate baseline	teammate experiment 1	teammate experiment 2
Architecture	Embedding 128, masked mean pooling, MLP (64), sigmoid output	Embedding 128, Conv1d widths 3/4/5 with 128 filters each, max-over-time pooling	Embedding 128, 2-layer BiGRU (128 per direction), additive attention pooling	Pending	Pending	Pending
Hyperparameters	AdamW lr 2e-3, batch 512, dropout 0.3, up to 8 epochs	AdamW lr 1e-3, batch 256, dropout 0.5, up to 5 epochs	AdamW lr 1e-3, batch 256, dropout 0.3, up to 5 epochs	Pending	Pending	Pending
Accuracy	0.9304	0.9467	0.9558	Pending	Pending	Pending
Accuracy 95% CI	[0.9276, 0.9330]	[0.9446, 0.9488]	[0.9537, 0.9577]	Pending	Pending	Pending
Precision (macro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Recall (macro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
F1 (macro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Precision (micro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Recall (micro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
F1 (micro)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Precision (weighted)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Recall (weighted)	0.9304	0.9467	0.9558	Pending	Pending	Pending
F1 (weighted)	0.9304	0.9467	0.9558	Pending	Pending	Pending
Confusion matrix [TN FP; FN TP]	[17,718 1,282; 1,363 17,637]	[17,951 1,049; 976 18,024]	[18,029 971; 710 18,290]	Pending	Pending	Pending
ROC-AUC	0.9800	0.9883	0.9917	Pending	Pending	Pending
PR-AUC	0.9802	0.9885	0.9920	Pending	Pending	Pending
MCC	0.8608	0.8934	0.9116	Pending	Pending	Pending
MCC 95% CI	[0.8553, 0.8659]	[0.8891, 0.8975]	[0.9076, 0.9155]	Pending	Pending	Pending
Brier score	0.0520	0.0399	0.0337	Pending	Pending	Pending
ECE (15 bins)	0.0045	0.0049	0.0110	Pending	Pending	Pending
McNemar vs baseline (b / c, p)	-(baseline)	613 / 1,233, p=4.67e-47	557 / 1,521, p=4.65e-99	Pending	Pending	Pending
Parameters	3,848,321	4,037,377	4,367,873	Pending	Pending	Pending
Training time (min)	0.3545	0.9865	19.1767	Pending	Pending	Pending
Epochs run (best)	8 (8)	5 (5)	5 (4)	Pending	Pending	Pending
Train examples/sec	279,909	54,074	2,271	Pending	Pending	Pending
Inference examples/sec	2,126,409	193,858	29,489	Pending	Pending	Pending
Peak GPU memory (MB)	1,149	1,277	1,392	Pending	Pending	Pending
Hardware	NVIDIA GeForce RTX 5090	NVIDIA GeForce RTX 5090	NVIDIA GeForce RTX 5090	Pending	Pending	Pending
Checkpoint (SHA-256)	725dff759e4	b4226830a1dc	2ecf44d666cf	Pending	Pending	Pending

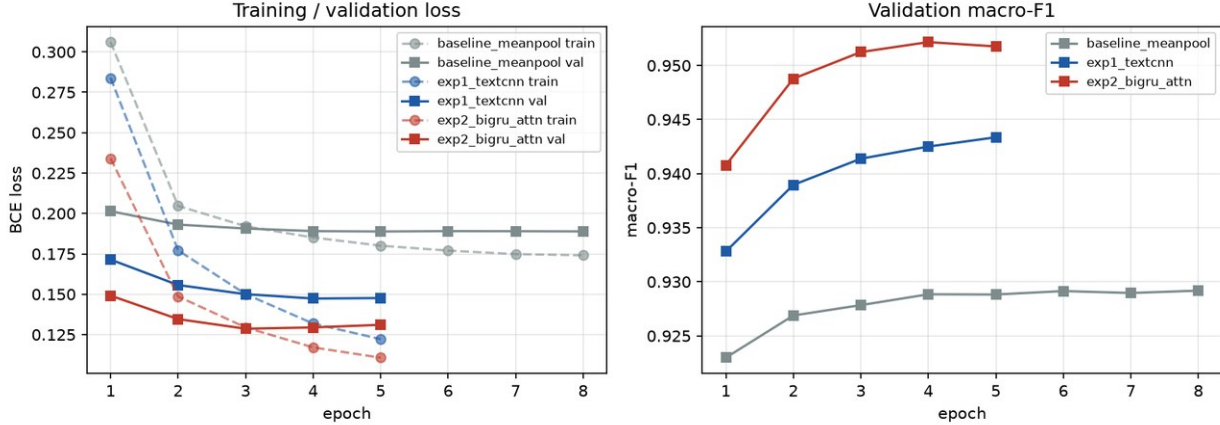
Source: task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849/comparison.csv, task2_sentiment/shared_eval/metrics.py

Macro, micro and weighted scores are identical because the test set is exactly balanced (19,000 reviews per class): weighted averaging then uses equal weights, and micro averaging always equals accuracy for a single-label binary task.

Table 4. Error rate (%) of Shubin's three models on subsets of the test set.

Slice (n)	Baseline	TextCNN	BiGRU + attn.
Contains a negation (28,544)	7.47	5.50	4.48
Contains a contrast word (22,713)	7.89	5.94	4.87
Neither negation nor contrast (6,688)	4.93	4.53	4.16
Short, at most 50 words (9,287)	6.96	5.29	5.01
Medium, 51 to 150 words (16,910)	6.94	5.22	4.09
Long, more than 150 words (11,803)	7.00	5.52	4.45

Source: task2_sentiment/shubin_thomas/metrics_report.csv

**Figure 2.** Training and validation loss and validation macro-F1 per epoch for the three Task 2 models (task2_sentiment/shubin_thomas/outputs/yelp3_20261001-182849/training_curves.png)

3.2 Discussion

Strengths. The ranking BiGRU > TextCNN > baseline holds for every discrimination metric, the 95% bootstrap intervals do not overlap, and paired McNemar tests on the same 38,000 reviews give p-values below $1e-20$ for each pair. Compared with the baseline, the BiGRU removes 36% of all errors, and 40% and 38% of the errors on reviews containing a negation or a contrast word, which is exactly where a bag of words cannot work. All three models are well calibrated (ECE at most 1.1%). Since the shared embedding table holds 3.84 of the roughly 4 million parameters in every model, the gains come from the way the text is read rather than from model size.

Weaknesses. The BiGRU is by far the slowest model: it trains about 120 times slower than the baseline (19 minutes against 21 seconds) and it is slightly over-confident, with 544 of its 1,681 errors made at a confidence above 90%. Short reviews are its hardest subset, and reviews with mixed sentiment account for most of its confident mistakes.

Limitations. The labels come from star ratings, not from the text, so some are simply wrong: 5 of the 20 errors we reviewed contradict their own text, and 932 test reviews are misclassified by all three models. Without pretrained knowledge, idioms and sarcasm are hard to learn. We used a single seed per model and a fixed decision threshold of 0.5.

Next steps. Marking the tokens that follow a contrast word or a negation is a cheap change we would test first, on the contrast slice with a McNemar test. We would also try a longer maximum length for long reviews, a small Transformer encoder trained from scratch, an ensemble of the three models with weights chosen on validation, temperature scaling, and several seeds per model.

3.3 Error Review (Shibin Thomas)

We reviewed 20 test reviews misclassified by the best model (BiGRU with attention): the five most confident false positives, the five most confident false negatives, five errors closest to the 0.5 threshold, and five errors from the subset with the highest error rate (short reviews). Each review was read and assigned an error type by hand. Ten of them are shown in Table 5.

Table 5. Examples from the manual review of 20 misclassified test reviews.

#	Label	P(pos)	Error type	Review text (excerpt)
2	neg	1.000	Label noise	<i>Wow love the place and everything is very clean and new! Great place to come and relax worth a try!</i>
4	neg	1.000	Mixed sentiment, implicit verdict	<i>About average so far as steakhouses go. My rib eye was very tasty but a little over cooked. ... Over all I'd say they are more for show than content. If you want to have a great high end steak I'd recommend Ruth Chris...</i>
6	pos	0.000	Label noise (edited review)	<i>EDIT: They really did change the service up since I last posted this. Horrible service. Used to be my favorite pizza in the city...</i>
9	pos	0.001	Sentiment target confusion	<i>Last night several parents came in with over 15 children ... The bartender expressed that it was not a place to have children running around...</i>
13	pos	0.498	Mixed sentiment, humour	<i>Their alcohol selection sucks ... dreamcrushed my cocktail dreams. ... However, the burger was nice, very juicy and rare.</i>
14	neg	0.502	Implicit negativity, domain shift	<i>Overly sentimental, non stop references to her kids, how blessed she is ... too many covers from elite performers...</i>
15	pos	0.497	Idiom	<i>the pizza was to my door in 15 minutes! ... the cookie's are to die for.</i>
16	neg	0.999	Faint praise	<i>Standard take out and it's cheap. Service is fast and friendly. I go here when I'm too lazy to walk to Zaw's...</i>
18	pos	0.002	Complex negation	<i>despite still not digging their ordering process, their food is just too good to disrespect with a 2 star review.</i>
19	pos	0.002	Sarcasm	<i>For being a DUMP, should expect much more. Flys, stink, garbage, dirt ... Keepin it real dumpy!</i>

Source: task2_sentiment/shibin_thomas/failure_analysis.md

Table 6. Error types among the 20 reviewed errors.

Error type	Count
Mixed sentiment (praise and complaints in the same review)	7
Label noise (text contradicts the star rating)	5
Faint praise or implicit negativity	3
Complex negation, target confusion, truncation, idiom, sarcasm	1 each

Five of the 20 errors are not really model errors, because the prediction agrees with the text and the star rating does not. The most common error the model could learn to avoid is mixed sentiment, where the verdict sits in the clause after “but” or “however”. As a testable fix we propose to prefix every token after the last contrast word with a marker, retrain the BiGRU with only this change, and compare the error rate on the contrast slice (currently 4.87%) with a McNemar test on the same test set. The teammate's error review is in their own task2_sentiment/ failure_analysis.md.

3.4 Evidence

Evidence	Location in the repository
Configurations	task2_sentiment/shibin_thomas/configs
Training and evaluation logs	reproducibility/raw_logs/task2_sentiment/shibin_thomas/yelp3_20261001-182849
Run manifest	reproducibility/manifests/task2_sentiment/shibin_thomas/yelp3_20261001-182849.json
All metrics for every model	task2_sentiment/shibin_thomas/metrics_report.csv
ROC, PR, reliability and slice plots	task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849
Learned embedding neighbours	task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849/embedding_neighbours.md
Checkpoints	task2_sentiment/shibin_thomas/checkpoints/yelp3_20261001-182849
Results and analysis	task2_sentiment/shibin_thomas/results.md
Executed notebook	task2_sentiment/shibin_thomas/src/task2_yelp_sentiment.ipynb

4 Task 3: CycleGAN Style Transfer between Monet Paintings and Photos

The goal is unpaired translation between 300 Monet paintings (domain A) and 7,038 photos (domain B) with a CycleGAN [3] trained from random initialisation. Shibin's generators are ResNet encoder-decoders with nine residual blocks; upsampling uses nearest-neighbour resizing followed by a convolution instead of transposed convolutions, to avoid checkerboard artifacts. The discriminators are 70x70 PatchGANs. Training combines a least-squares adversarial loss, a cycle-consistency loss (weight 10) and an identity loss (weight 5). The Kaggle score is computed with the instructor's script: FID and the script's MiFID on the first 300 sorted images in each direction, averaged, and the final score is the mean of the two values (lower is better). The submitted images are the unedited outputs of the trained generators; the pretrained Inception and LPIPS networks are only used to measure them.

4.1 Model Comparison

Table 7. Task 3 comparison. Directional metrics use the 300 scored images; the Kaggle score is (FID + MiFID) / 2. Run `cyclegan_v1_20261001-201801`.

	Shibin: photo to Monet	Shibin: Monet to photo	teammate CycleGAN
Architecture	Generators: ResNet-9 (c7s1-64, two stride-2 convs, 9 residual blocks, two resize-conv layers, tanh), InstanceNorm, reflection padding, 11.38M parameters each. Discriminators: 70x70 PatchGAN, 2.76M each.		Pending
Hyperparameters	LSGAN, cycle weight 10, identity weight 5; Adam 2e-4 (0.5, 0.999); 20 epochs constant LR then 20 epochs linear decay, 2,000 unpaired pairs per epoch; batch 4; image pool 50; resize 286, random crop 256, flip		Pending
FID	118.07	117.34	Pending
MiFID (instructor script)	0.4105	0.4330	Pending
KID	0.0197	0.0362	Pending
Precision (k = 3)	0.267	0.607	Pending
Recall (k = 3)	0.640	0.320	Pending
Density / coverage (k = 5)	0.221 / 0.523	0.836 / 0.757	Pending
Cycle reconstruction L1	0.0622	0.0531	Pending
LPIPS, input vs translation	0.451	0.366	Pending
LPIPS, input vs reconstruction	0.384	0.468	Pending
Content cosine similarity	0.750	0.794	Pending
Kaggle file: FID / MiFID (average)	117.7069 / 0.4217, score 59.06		Pending
Kaggle leaderboard: score / rank	Pending / Pending		Pending
Final epoch losses: G / D_A / D_B	4.458 / 0.089 / 0.139		Pending
Final epoch cycle A / B, identity A / B	0.096 / 0.120, 0.108 / 0.110		Pending
Gradient norm G (mean / max), D (mean / max)	48.3 / 727, 17.1 / 342		Pending
Non-finite steps / total steps	0 / 20,000		Pending
Training time / throughput / peak GPU memory	91.2 min / 36.0 images/s / 19,922 MB		Pending
Total parameters	28,285,832		Pending
Human audit (2 raters, 30 samples)	Pending		Pending
Hardware	NVIDIA GeForce RTX 5090		Pending
Checkpoints (SHA-256)	G_BA.pt e00ed3819109	G_AB.pt 4185c02deae2	Pending

Source: task3_gan/shibin_thomas/full_metrics_report.csv, task3_gan/shibin_thomas/submission.csv, task3_gan/shibin_thomas/kaggle_leaderboard.json

4.2 Training Behaviour and Results

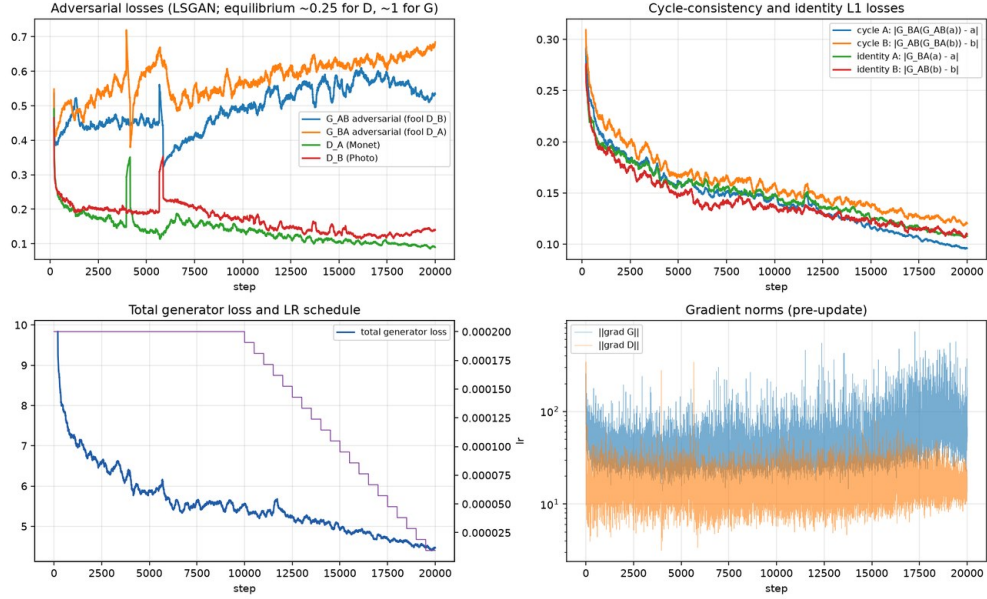


Figure 3. Task 3 adversarial, cycle and identity losses, total generator loss with the learning-rate schedule, and gradient norms over 20,000 steps (task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801/loss_curves.png)

Table 8. FID on the 300 scored images during training (monitoring only).

Epoch	5	10	15	20	25	30	35	40
FID, photo to Monet	153.2	145.4	147.1	128.7	130.2	119.0	123.5	121.6
FID, Monet to photo	187.7	155.2	169.5	128.6	119.7	121.8	121.3	118.1

Source: reproducibility/raw_logs/task3_gan/shibin_thomas/cyclelegan_v1_20261001-201801/epochs.csv

Both discriminators settle below the least-squares balance point of 0.25 and keep improving slowly, while the generators' adversarial losses rise; by the end the Monet discriminator (loss 0.089) is clearly winning, a sign that it has started to memorise the 300 paintings. The cycle and identity losses fall smoothly over the whole run. FID improves quickly during the first 25 epochs and then stays within about 4 points of 120, which is within the noise of a 300-image FID. No training step produced a NaN or infinite value.

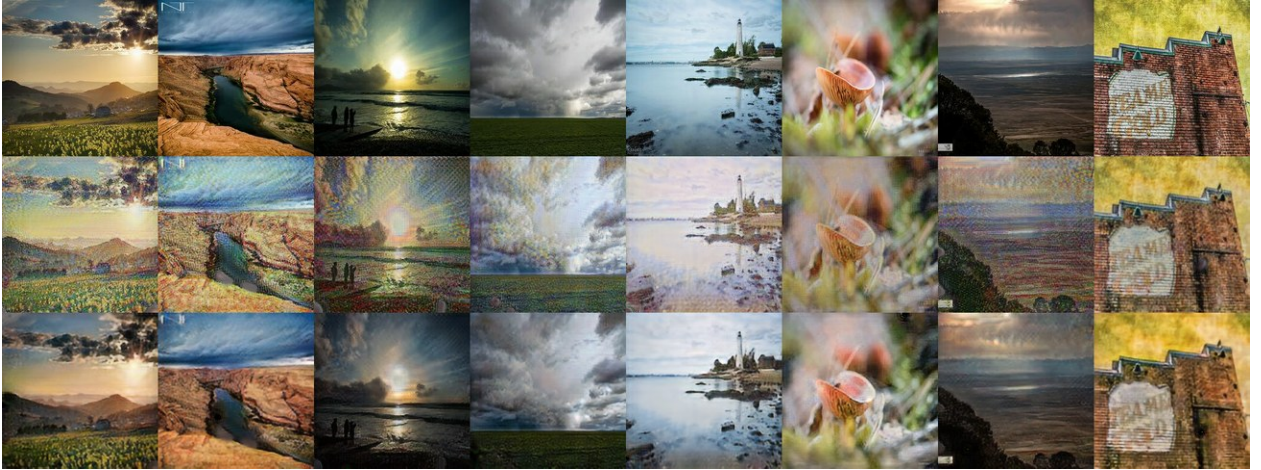


Figure 4. Photo to Monet. Top row: input photos; middle: generated paintings; bottom: reconstructions (task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801/final_samples_B2A.png)



Figure 5. Monet to photo. Top row: input paintings; middle: generated photos; bottom: reconstructions (task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801/final_samples_A2B.png)

4.3 Discussion

Strengths. Training was stable from start to finish and neither generator collapsed. The cycle constraint works: reconstructions keep the layout, objects and colours of the inputs (cycle L1 of 0.05 to 0.06), and photo-to-Monet translations preserve the content of the scene (content cosine 0.75) while moving the colours towards Monet's palette. Our implementation of the scoring protocol matches the instructor's script, so the submitted numbers can be verified.

Weaknesses. Only 27% of the generated paintings fall inside the feature manifold of real Monets (precision 0.27). The main reasons are a regular stippled texture that replaces Monet's brush strokes and the washing out of dark photos. In the other direction the generated photos cover only a third of the photo distribution (recall 0.32) and still look painted. The Monet discriminator over-fits the small training set.

Limitations. FID on 300 images has high variance, and we trained a single seed. The script's MiFID is a mean cosine distance between unrelated real and generated images and hardly changes between models, so the Kaggle score is driven almost entirely by FID. The human audit is still being completed.

Next steps. Based on these findings we prepared a second configuration (configs/cyclelegan_v2.yaml) with differentiable augmentation for the discriminators [18], an exponential moving average of the generator weights, a lower identity weight of 2.5, 60 epochs, and selection of the exported epoch on held-out photos. A multi-scale discriminator and a low-frequency content loss are further options.

4.4 Failure Analysis (Shibin Thomas)

We ranked all translations by cycle error, by content similarity and by the distance to the nearest real image of the target domain, and inspected the worst four images for each criterion. In each figure below the columns show the input (top), the translation (middle) and the reconstruction (bottom).



Figure 6. Dark and night photos translated to Monet style
(task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801/failure_B2A_nn_dist.png)

Dark photos. Low-light scenes such as a river at dusk or a camel silhouette against a sunset are turned into bright, washed-out images covered in coloured speckles, and silhouettes lose their edges. These are the least realistic outputs. The Monet set contains almost no night scenes, so the generator brightens every image. Colour augmentation of the discriminator's inputs is the fix we included in the second configuration.

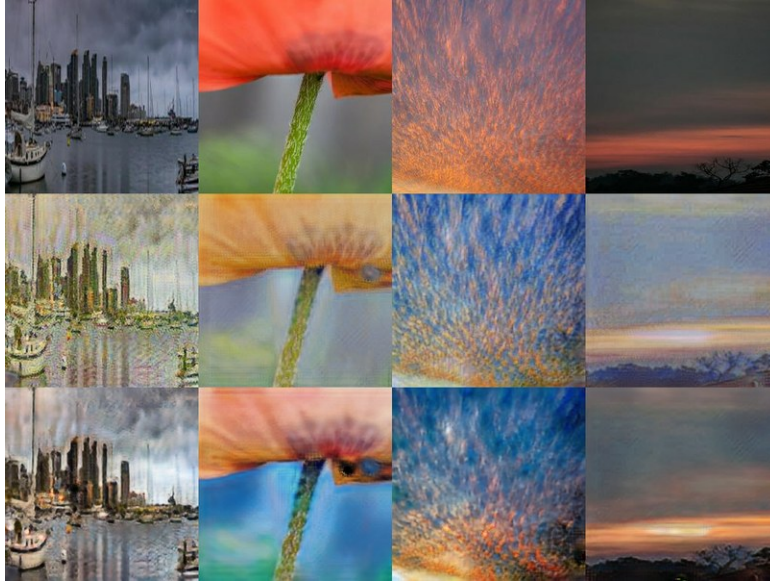


Figure 7. Photos with the highest cycle-reconstruction error
(task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801/failure_B2A_cycle_l1.png)

Recoloured warm tones. Strong reds and oranges are replaced by Monet's blues and lilacs: an orange sky becomes blue and a red poppy turns pale orange. The reconstructions nevertheless recover the original colours, which means the generator hides the information in a faint signal that the cycle loss can read back, an effect described by Chu et al. [17]. When this hidden signal is not enough, the cycle error is highest (0.13 to 0.17, about 2.5 times the average).



Figure 8. Monet paintings with the lowest content similarity after translation to photos
(task3_gan/shibin_thomas/outputs/cycleGAN_v1_20261001-201801/failure_A2B_content_cos.png)

Foggy paintings. Pale, low-contrast paintings of fog on the Thames become saturated orange sunsets or turquoise storms, and a thin poplar tree disappears. Photos are rarely foggy, so the generator invents contrast and colour. We also observed a regular stipple texture in most photo-to-Monet outputs and visible brush strokes and signatures in most Monet-to-photo outputs. The full analysis of all five failure types is in `failure_analysis.md`; the teammate's analysis is in their own Task 3 folder.

4.5 Evidence

Evidence	Location in the repository
Configuration	task3_gan/shibin_thomas/configs/cycleGAN_v1.yaml
Training and evaluation logs	reproducibility/raw_logs/task3_gan/shibin_thomas/cycleGAN_v1_20261001-201801
Run manifest	reproducibility/manifests/task3_gan/shibin_thomas/cycleGAN_v1_20261001-201801.json
Submitted predictions	task3_gan/shibin_thomas/outputs
Kaggle submission file	task3_gan/shibin_thomas/submission.csv
Plots and sample grids	task3_gan/shibin_thomas/outputs/cycleGAN_v1_20261001-201801
Generator checkpoints	task3_gan/shibin_thomas/checkpoints/cycleGAN_v1_20261001-201801
Human audit material	task3_gan/shibin_thomas/outputs/human_audit
Results and analysis	task3_gan/shibin_thomas/results.md
Failure analysis	task3_gan/shibin_thomas/failure_analysis.md
Executed notebook	task3_gan/shibin_thomas/src/task3_cycleGAN.ipynb

5 Pre-Submission Checklist

Item	Status
Every member's folder exists under all three tasks with code, configuration, logs and results.md	Shibin: done; teammate: pending
No two members share the same architecture and hyperparameters	To be confirmed with the teammate's entries
Task 1 uses no prebuilt Transformer or attention modules	Done (checked by a unit test)
Task 2 uses no pretrained embeddings or language models; three models per member	Shibin: done; teammate: pending
All required metrics are reported for every model in Tasks 1 to 3	Shibin: done; teammate: pending
Task 3 Kaggle submission made and leaderboard rank recorded	Submitted; rank to be recorded
Comparison tables with architecture, hyperparameters and metrics for all three tasks	Done (Tables 1, 3 and 7)
Raw logs and manifests committed and unmodified	Done

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] R. Eldan and Y. Li. TinyStories: How small can language models be and still speak coherent English? *arXiv:2305.07759*, 2023.
- [3] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *ICCV*, 2017.
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- [5] R. Xiong et al. On layer normalization in the Transformer architecture. In *ICML*, 2020.
- [6] X. Zhang, J. Zhao, and Y. LeCun. Character-level convolutional networks for text classification. In *NeurIPS*, 2015.
- [7] Y. Kim. Convolutional neural networks for sentence classification. In *EMNLP*, 2014.
- [8] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov. Bag of tricks for efficient text classification. In *EACL*, 2017.
- [9] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- [10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.
- [11] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017.
- [12] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley. Least squares generative adversarial networks. In *ICCV*, 2017.
- [13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *NeurIPS*, 2017.
- [14] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton. Demystifying MMD GANs. In *ICLR*, 2018.
- [15] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila. Improved precision and recall metric for assessing generative models. In *NeurIPS*, 2019.
- [16] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [17] C. Chu, A. Zhmoginov, and M. Sandler. CycleGAN, a master of steganography. *NeurIPS Workshop*, 2017.
- [18] S. Zhao, Z. Liu, J. Lin, J.-Y. Zhu, and S. Han. Differentiable augmentation for data-efficient GAN training. In *NeurIPS*, 2020.